Here is a design specification for an agentic application which queries four language models with the same query.

Language Models

- To start: Gpt 5.1, Sonnet 4.5, Gemini 3, Grok 4.1.
- For now, I can access gpt and grok through my azure account.
- We can access Sonnet and Gemini through subscriptions that I have with them.
- Each Model Agent encapsulates its LLM API call as an internal tool, maintaining separation between agent orchestration logic and model invocation.

User Interface (see wireframe below)

- At the top, render 4 cells across two rows, each with two columns, with each cell containing the results from one of the models.
- Each cell should contain a read-only, scrollable text-area.
- Each cell should have a copy to clipboard in markdown format button.
- In the middle, render another read-only, scrollable text area that stretches across the page horizontally.
- Add a copy to clipboard in markdown format button.
- At bottom, render a prompt text-area HTML element that stretches across the screen.
- Include a Search Button and Clear Button.

Prompt Clarification Step

- When the search button is invoked, the supervisor agent will receive and record the initial prompt:

Supervisor receives: { user_prompt: "Explain quantum computing", mode: "search

Searching

- The supervisor will then fan-out the query to each model child agent for parallel inferencing passing state as a structured JSON object:

{ refined_prompt: "...", conversation_id: "abc-123", role: "responder" // vs "debater" later }

- Each model child agent will load its prompt template
- Call its model
- Maintain chat history for each turn, reasoning logic, and other state in its own local state store:

GPT Agent state: { agent_id: "gpt", conversation_history: [{ role: "user", content: "refined prompt..." }, { role: "assistant", content: "my response..." }]}

- The model child agent will also stream its response back to the Supervisor as a structured JSON object.
- The Supervisor will collect child agent state in a structured JSON object:

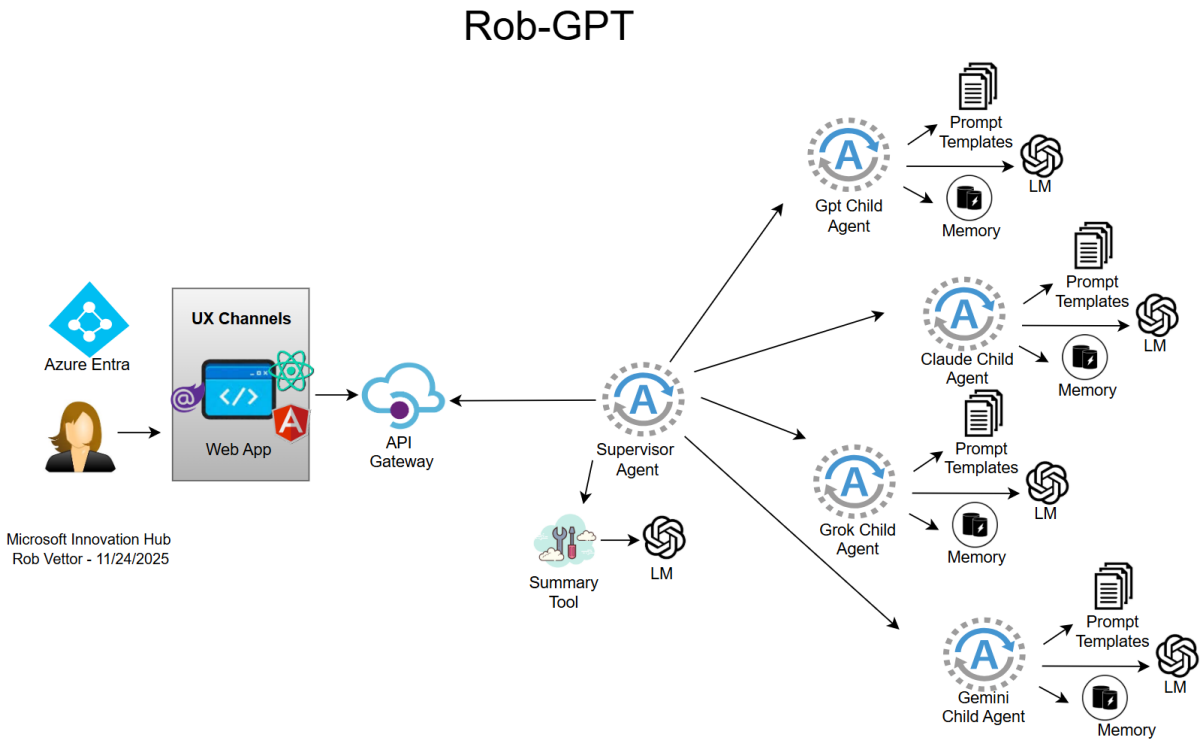
Supervisor state: { conversation_id: "abc-123", original_prompt: "...", refined_prompt: "...", responses: { gpt: { text: "...", status: "complete" }, claude: { text: "...", status: "complete" }, gemini: { status: "pending" }, grok: { status: "pending" } }

- Upon completion, the orchestrator agent will the completion to the corresponding UI widget.
- Upon completion of all child agents, the orchestrator agent will send all child agent responses to the Summary tool, which synthetize and summarize responses and output to the summary UX.
- The local state store for each agent will be abstracted in an interface allowing for different state stores to plug-in dynamically.
- To be clear, two copies of each model child agent response will be maintained:

Two Copies of Response Data

Location	What it holds	Why
Agent	Its own conversation history	Needs context for coherent debate responses
Supervisor	Aggregated responses from all agents	Needs full picture for routing, summarization, UI streaming

<u>Rules of the Road</u>

- All agent/tool communication is encapsulated in structured JSON. No native strings.
- Nothing is hard-coded. Use interface and a provider model for everything.

- When the copy button is invoked, the targeted agent will return state in to the user's clipboard in MarkDown format.
- Please wrap all code with structured exception handling.
- Please capture telemetry using Azure Application Insights.
- Please implement authentication against Azure Entra implementing the Azure Default Credential constructs.
- Each agent will include a prompt template with placeholders to inject the prompt snippets passing by the Supervisor agent.